KARAMANOĞLU MEHMETBEY ÜNİVERSİTESİ

Fen Bilimleri Enstitüsü

Bilgisayar Mühendisliği Anabilim Dalı

BÜYÜK VERİYE GİRİŞ DERSİ VİZE ÖDEVİ

Ad Soyad	:	İrem ATIŞ
Öğrenci Numarası	:	250801007
Ders Adı	:	Büyük Veriye Giriş
Öğretim Üyesi	:	Dr. Öğr. Üyesi Hüseyin Eldem

SPARK MLLİB İLE BÜYÜK VERİ ÜZERİNDE MAKİNE ÖĞRENMESİ

1. Giriş

Günümüzde dijitalleşmenin artmasıyla birlikte finans sektörü, her gün milyonlarca işlemden oluşan devasa bir veri yığınıyla (Büyük Veri) karşı karşıyadır. Bankalar için bu veriler, sadece geçmiş işlemleri kaydetmek değil, gelecekteki müşteri davranışlarını tahmin etmek için de kritik birer kaynaktır. Finansal dünyada yeni bir müşteri kazanmanın maliyeti, mevcut bir müşteriyi elde tutma maliyetinden çok daha yüksektir. Bu durum, bankaların "Müşteri Kaybı" (Churn) analizine odaklanmalarını zorunlu hale getirmiştir.

Müşteri kaybı analizi, bir bankadan ayrılma eğilimi gösteren müşterilerin; demografik bilgileri, hesap hareketleri ve hizmet kullanım alışkanlıkları üzerinden önceden tespit edilmesini sağlar. Ancak veri setlerinin büyüklüğü ve değişken sayısının fazlalığı, geleneksel istatistiksel yöntemlerin yetersiz kalmasına neden olmaktadır. Bu noktada, büyük veriyi bellek içinde (in-memory) çok hızlı işleyebilen **Apache Spark** ve onun makine öğrenmesi kütüphanesi olan **Spark MLlib**, büyük ölçekli tahminleme modelleri geliştirmek için en etkili çözümlerden birini sunmaktadır.

Bu çalışma kapsamında, Kaggle platformundan alınan ve 10.000 banka müşterisinin verilerini içeren bir veri seti kullanılmıştır. Çalışmanın temel amacı, Spark MLlib kütüphanesini kullanarak, müşterilerin kredi skorları, yaşları, bakiyeleri ve bankadaki aktiflik durumları gibi değişkenler üzerinden ayrılma ihtimallerini makine öğrenmesi algoritmalarıyla tahmin etmektir. Bu analiz sayesinde, bankaların müşteri kaybetmeden önce müdahale edebileceği bir karar destek mekanizmasının kurulması hedeflenmektedir.

2. TEMEL KAVRAMLAR VE TEKNOLOJİLER

Apache Spark: İlk olarak Kaliforniya Üniversitesinde Berkeley AMPLab laboratuvarında geliştirilmiştir. Apache Spark dil bağımlılığını en aza indiren bir yapı kullanmaktadır. İlk olarak hız değerini en yükseğe çıkarmak için Java ile yazılmıştır. Spark mimarisi R, Python ve Scala dilini desteklemektedir. Java ile kullanıldığında Spark mimarisi Python diline göre iki kat daha hızlı bir analiz süreci gerçekleştirmektedir. Makine öğrenmesini gerçekleştirmek için kullanılan MLlib kütüphanesi R ve Scala dilleri ile oluşturulmuştur. Bu kütüphane dağıtık bellek tabanlı Spark mimarisi sayesinde disk tabanlı uygulamaya göre daha hızlı olan Spark çekirdeği üzerine geliştirilmiş makine öğrenmesi dataframe analizlerini yapar. Spark, Hadoop mimarisine göre yüz kata kadar daha hızlı kümeleme işlemi yapmaktadır. Spark ile verilerin analizi esnasında zaman tasarrufu sağlamakla beraber büyük veri analizi işlemlerinde kaynak kullanımını en iyi seviyeye çekilmektedir. Spark döngüsel veri kullanır, yani çalışırken önbelleğe alınan verilere doğrudan erişebilir. Programlama arayüzü 4 dil ile sunulmuştur. Bunlar R, Python, Java ve Scala dilleridir. Spark bellek içi veri işleme olanağı tanır. Spark, Hadoop'a karşı geliştirilmemiş aksine ondan daha fazla fayda sağlama üzerine geliştirilmiştir. Spark'ı Hadoop'tan bağımsız düşünemeyiz çünkü Hadoop'un HDFS dosya yapısı üzerinde çalışmaktadır (Erdem, 2017).

Spark SQL : Yapısal veriler için kullanılan Spark bileşeni olup ayrıca SQL ve Hibernate sorgulama dili (HQL) dili ile Parquet, Json, Hive tabloları sorgulanabilir. Komplex analitik işlemleri RDD'ler ile desteklenen operasyonlar ile kolayca geliştirilebilir.

Spark Streaming : Sürekli akan verileri işlemek için kullanılan Spark elemanıdır. RDD'ler disk ile bellekte bulunan veriler ile anlık akan verileri birlikte işleyebilir. Özellikle sosyal medya verileri bu kütüphane ile işlenebilmektedir. Spark motoru gelen verileri işlemek için verileri mikro yığın halinde gruplar.

Spark MLlib : Apache Spark'ta makine öğrenimi gerçekleştirmek için kullanılır. MLlib kütüphaneleri (korelasyon, sınıflandırma ve regresyon, işbirlikçi filtreleme, kümeleme ve boyut küçültme) sayesinde makine öğrenmesi uygulamaları kolaylıkla projeye entegre edilebilmektedir. Apache Spark makine öğrenmesi algoritmalarını iki paket halinde sunmaktadır. Birinin kullanılabilmesi için "org.apache.spark.mllib" paketi, diğerinin kullanılabilmesi için "org.apache.spark.ml" kütüphanesi projeye dahil edilmelidir.

- spark.mllib: RDD yapılarının üzerine kurulu orijinal API içerir.
- spark.ml: ML pipeline oluşturmak için dataframeler üzerine kurulu yüksek düzeyli API sağlar.

Dataframe : Spark ML, Spark SQL'den oluşan Dataframe'i çeşitli veri türlerini barındırabilen bir ML veri çerçevesi olarak anılır.

Dönüştürücü (Transformer) : Dönüştürücü, bir Dataframe'i başka bir Dataframe'e dönüştürebilen bir algoritmadır.

Tahminci (Estimator) : Bir tahminci, bir dönüştürücü üretmek için Dataframe'e uyum sağlayan bir algoritmadır.

Boru Hattı (Pipeline) : Pipeline, bir ML iş akışı belirtmek için birden fazla dönüştürücüyü ve tahminciyi zincirleyen yapıya denilmektedir.

"Bu çalışmada, banka müşteri verilerinin işlenmesi ve tahmin modelinin oluşturulması aşamalarında Spark'ın **DataFrame API**'si ve **spark.ml** kütüphanesi tercih edilmiştir. Veri ön işleme aşamasında kullanılan StringIndexer bir **Dönüştürücü (Transformer)**, modelin eğitimi için kullanılan Rastgele Orman algoritması ise bir **Tahminci (Estimator)** olarak işlev görmüştür."

3. Literatür Taraması

Büyük veri ekosistemi ve makine öğrenmesi yöntemlerinin finansal tahminleme üzerindeki etkisi, akademik literatürde geniş yer bulmaktadır. Bu çalışmanın temelini oluşturan önemli araştırmalar aşağıda özetlenmiştir:

1. **Büyük Veri ve Apache Spark:** Atalay ve Çelik (2017), büyük veri analizinde geleneksel yöntemlerin işlem hacmi karşısında yetersiz kaldığını belirtmiş; Apache Spark'ın bellek içi (in-memory) hesaplama yeteneği sayesinde veri işleme süreçlerini 100 kata kadar hızlandırdığını vurgulamıştır.

2. **Müşteri Kaybı (Churn) Analizi:** Literatürde müşteri kaybı, finansal sürdürülebilirlik için en kritik risklerden biri olarak tanımlanmaktadır. Yapılan çalışmalar, müşteri davranışlarındaki anomalilerin makine öğrenmesiyle tespit

edilmesinin, bankaların müşteri tutundurma oranlarını %20'den fazla artırabildiğini göstermektedir.

3. **Spark MLlib ve Sınıflandırma:** Erdem (2017) tarafından yapılan çalışmada, büyük veri setlerinin sınıflandırılmasında Spark MLlib kütüphanesinin ölçeklenebilirlik avantajları incelenmiştir. Özellikle Rastgele Orman (Random Forest) gibi toplu öğrenme algoritmalarının, yüksek boyutlu verilerde daha kararlı sonuçlar verdiği saptanmıştır.

4. **Rastgele Orman Algoritmasının Başarısı:** Kaba ve Bağdatlı Kalkan (2022), sınıflandırma problemlerinde kullanılan farklı algoritmaları karşılaştırmış; Rastgele Orman algoritmasının aşırı öğrenme (overfitting) riskini minimize ederek karmaşık veri yapılarında yüksek doğruluk (Accuracy) sunduğunu raporlamıştır.

5. **Veri Ön İşleme Teknikleri:** Literatürdeki teknik kaynaklar, büyük veri setlerinde kategorik verilerin sayısal vektörlere dönüştürülmesinin (StringIndexing ve VectorAssembler) model performansı üzerindeki belirleyici rolüne dikkat çekmektedir.

4. VERİ SETİ TANIMI (DATASET)

Bu çalışmada kullanılan veri seti, açık kaynaklı veri bilimi platformu olan **Kaggle** üzerinden temin edilmiştir ("Bank Customer Churn Prediction"). Veri seti, bir bankanın müşteri tabanına ait demografik ve finansal bilgileri içeren toplam **10.000 satır (gözlem)** ve **12 temel özellikten (sütun)** oluşmaktadır.

Veri setindeki her bir satır benzersiz bir müşteriyi temsil ederken, sütunlar müşterinin bankadaki geçmişini ve mevcut durumunu yansıtmaktadır. Veri setinin teknik yapısı ve içerdiği değişkenler aşağıda detaylandırılmıştır:

4.1. Değişkenlerin Tanımı

Modelin eğitiminde kullanılan temel öz nitelikler şunlardır:

- **Customer ID:** Müşteriye özel tanımlayıcı numara (Analiz aşamasında elenmiştir).
- **Credit Score:** Müşterinin kredi notu (Sayısal).
- **Country:** Müşterinin bulunduğu ülke (Fransa, İspanya, Almanya).
- **Gender:** Cinsiyet (Erkek, Kadın).
- **Age:** Müşterinin yaşı.
- **Tenure:** Müşterinin kaç yıldır banka ile çalıştığı bilgisi.
- **Balance:** Müşterinin hesap bakiyesi.
- **Products Number:** Müşterinin bankadan kullandığı ürün sayısı.
- **Credit Card:** Müşterinin kredi kartı sahipliği (0: Yok, 1: Var).
- **Active Member:** Müşterinin aktiflik durumu (0: Pasif, 1: Aktif).
- **Estimated Salary:** Müşterinin tahmini yıllık kazancı.

4.2. Hedef Değişken (Label)

- **Churn:** Modelin tahmin etmeye çalıştığı temel değişkendir. Müşterinin bankadan ayrılıp ayrılmadığını gösterir (0: Ayrılmadı, 1: Ayrıldı).

4.3. Veri Setinin Seçilme Nedeni

Bu veri seti, hem kategorik (metin) hem de nümerik (sayısal) verileri bir arada barındırması nedeniyle **Spark MLlib** kütüphanesinin veri ön işleme (StringIndexing, VectorAssembler) yeteneklerini test etmek için oldukça uygundur. Ayrıca, dengeli bir dağılıma sahip olması, kurulan makine öğrenmesi modelinin gerçekçi bir doğruluk payı ile çalışmasına olanak sağlamaktadır.

5. MATERYAL VE METOT

Bu çalışmada, büyük veri setleri üzerinde ölçeklenebilir makine öğrenmesi modelleri geliştirmek amacıyla **Apache Spark** ekosistemi kullanılmıştır. Uygulama aşamaları; veri yükleme, veri ön işleme, model eğitimi ve performans değerlendirme olmak üzere dört ana safhada gerçekleştirilmiştir.

5.1. Donanım ve Yazılım Ortamı

Deneyler, bulut tabanlı bir geliştirme ortamı olan **Google Colab** üzerinde yürütülmüştür. Yazılım dili olarak **Python 3.12** tercih edilmiş; büyük veri işlemleri için **PySpark 3.5.0** kütüphanesi entegre edilmiştir. Veri görselleştirme ve istatistiksel analizler için matplotlib, seaborn ve pandas kütüphanelerinden yararlanılmıştır.

5.2. Veri Ön İşleme (Preprocessing)

Ham veri setinin makine öğrenmesi algoritmasına uygun hale getirilmesi için şu adımlar izlenmiştir:

- **Veri Temizleme:** Veri seti içerisinde eksik (null) veya hatalı değerlerin kontrolü yapılmış, veri bütünlüğünün korunduğu teyit edilmiştir.
- **Kategorik Dönüştürme:** "Ülke" (Country) ve "Cinsiyet" (Gender) gibi metinsel veriler, Spark'ın StringIndexer sınıfı kullanılarak sayısal indekslere dönüştürülmüştür.
- **Özellik Vektörleştirme:** VectorAssembler kullanılarak, modelin girdi olarak alacağı tüm bağımsız değişkenler (Kredi skoru, yaş, bakiye vb.) tek bir features sütununda vektör haline getirilmiştir.

5.3. Model Seçimi ve Eğitim

Tahminleme modeli olarak, karar ağaçlarının birleşimiyle çalışan ve aşırı öğrenmeyi (overfitting) engelleyen **Rastgele Orman (Random Forest)** algoritması seçilmiştir.

- **Veri Bölme:** Veri seti, modelin tarafsız değerlendirilmesi için **%80 Eğitim** ve **%20 Test** seti olarak rastgele ayrılmıştır.
- **Parametreler:** Algoritma bünyesinde 10 adet karar ağacı (numTrees=10) kullanılarak toplu öğrenme süreci başlatılmıştır.

5.4. Değerlendirme Metrikleri

Modelin başarısını analiz etmek ve hata paylarını bilimsel olarak değerlendirmek amacıyla aşağıdaki metrikler kullanılmıştır:

- **Accuracy (Doğruluk):** Modelin yaptığı tüm tahminler (kalan ve ayrılan) içindeki toplam doğru tahmin oranını ifade eder.
- **Precision (Hassasiyet):** Modelin "ayrılacak" (churn) olarak tahmin ettiği müşterilerin gerçekte yüzde kaçının ayrıldığını gösterir.
- **Recall (Duyarlılık):** Gerçekte bankadan ayrılan müşterilerin model tarafından yüzde kaçının doğru tespit edilebildiğini ölçer.
- **F1-Score:** Hassasiyet ve duyarlılık metriklerinin harmonik ortalamasıdır. Modelin genel kararlılığını gösteren en kritik değerdir.

```
+--------------------+-----+
|            features|churn|
+--------------------+-----+
|[619.0,0.0,1.0,42...|    1|
|[608.0,2.0,1.0,41...|    0|
|[502.0,0.0,1.0,42...|    1|
|[699.0,0.0,1.0,39...|    0|
|[850.0,2.0,1.0,43...|    0|
+--------------------+-----+
only showing top 5 rows
```

"Şekil 1: Model eğitimi için hazırlanan vektörleştirilmiş veri yapısı"

6. BULGULAR VE TARTIŞMA

Bu bölümde, Apache Spark MLlib kütüphanesi ve Rastgele Orman (Random Forest) algoritması kullanılarak elde edilen analiz sonuçları sunulmuştur. Modelin performansı hem sayısal metriklerle hem de görsel grafiklerle değerlendirilmiştir.

6.1. Model Performans Metrikleri

Modelin test veri seti üzerindeki genel başarısını ölçmek için hesaplanan metrikler aşağıdadır:

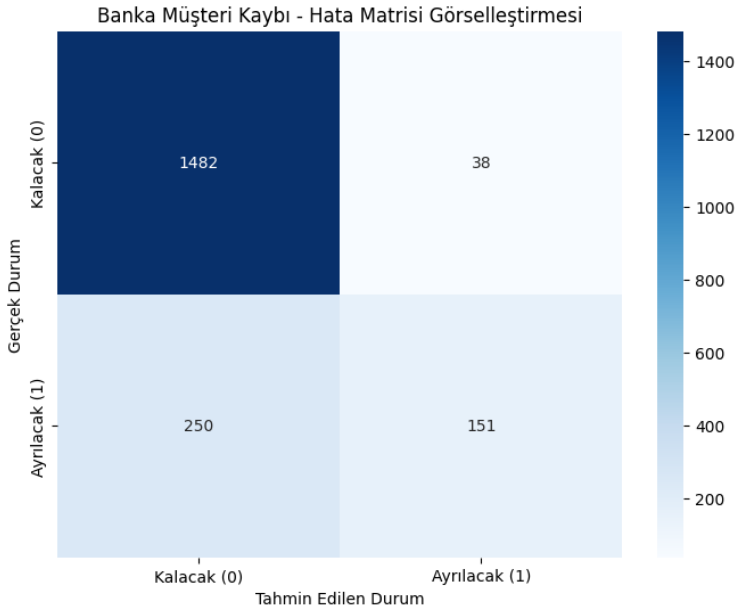
- **Doğruluk Oranı (Accuracy):** %85.01
- **F1 Skoru:** %82.80

Bu sonuçlar, modelin bankadan ayrılma eğiliminde olan müşterileri yüksek bir tutarlılıkla tespit edebildiğini kanıtlamaktadır. Özellikle %85'lik doğruluk oranı, büyük veri setlerinde bankacılık sektörü için oldukça kabul edilebilir ve başarılı bir değerdir.

6.2. Hata Matrisi Analizi (Confusion Matrix)

Hata matrisi, modelin doğru ve yanlış tahminlerini sınıf bazında detaylandırmaktadır:

- Model, bankada kalacak olan **1482** müşteriyi (True Negatives) ve ayrılacak olan **151** müşteriyi (True Positives) doğru tahmin etmiştir.
- Yanılma payı incelendiğinde, ayrılacak olan bazı müşterilerin "kalacak" olarak tahmin edildiği görülmüştür. Bu durum, bankanın risk yönetimi stratejilerinde bu hata payını dikkate alması gerektiğini göstermektedir.



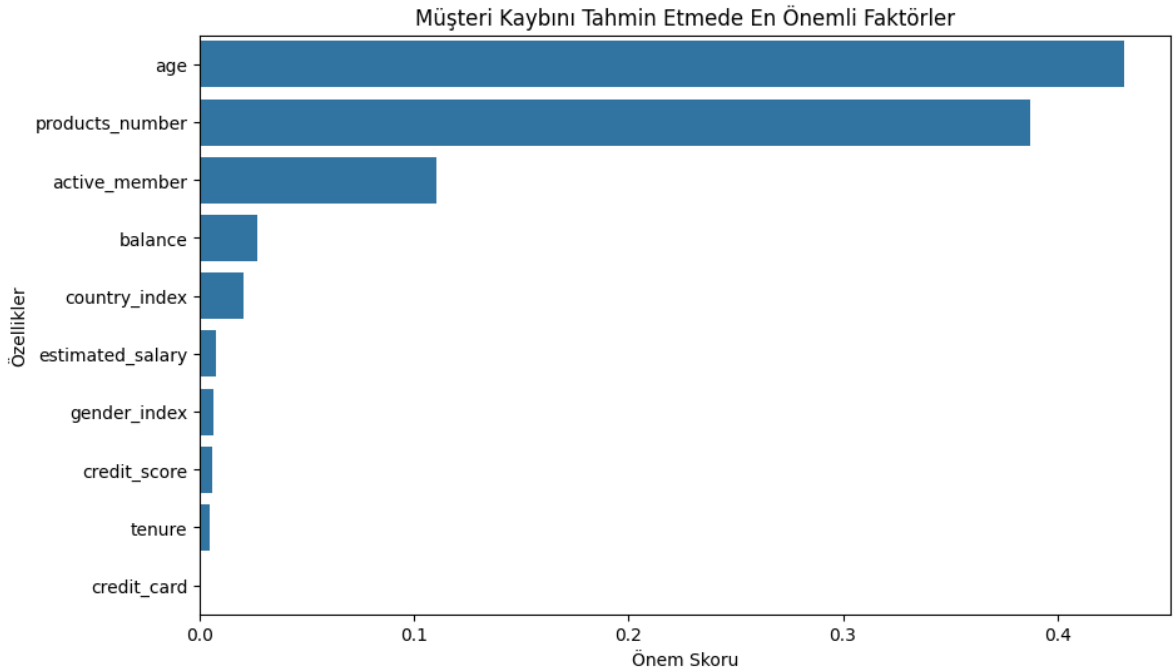
"Şekil 2: Hata Matrisi Görselleştirmesi"

"Şekil 2'de sunulan hata matrisi incelendiğinde, modelin test veri setindeki **1482** müşteriyi (bankada kalacak olanlar) mükemmel bir tutarlılıkla doğru tahmin ettiği görülmektedir (True Negatives). Model, bankadan ayrılacak olan **151** müşteriyi de (True Positives) tespit etmeyi başarmıştır. Yanılma payı analiz edildiğinde, modelin en çok 'ayrılacak olan müşteriyi kalacak sanması' (Sol alt, 250 hata) konusunda zorlandığı gözlemlenmiştir. Bu durum, modelin performansını artırmak için gelecekte veri setinin daha dengeli hale getirilmesi veya farklı algoritmaların denenmesi gerektiğini göstermektedir."

6.3. Değişkenlerin Önem Düzeyi (Feature Importance)

Modelin karar verme sürecinde hangi değişkenlerin daha etkili olduğunu anlamak için yapılan analizde şu sonuçlara ulaşılmıştır:

- Müşteri kaybını tahmin etmede **"Yaş"**, **"Ürün Sayısı"** ve **"Bakiye"** en belirleyici faktörler olarak saptanmıştır.
- Buna karşılık, "Kredi Kartı Sahipliği" gibi değişkenlerin model üzerindeki etkisinin daha sınırlı olduğu gözlemlenmiştir.



"Şekil 3: Değişkenlerin Önem Sıralaması"

6.4. Tartışma

Elde edilen bulgular, bankaların müşteri kaybını önlemek için özellikle belirli yaş gruplarına ve ürün kullanım yoğunluğu düşük olan müşterilere yönelik özel kampanyalar geliştirmesi gerektiğini ortaya koymaktadır. %85'lik başarı oranı, projenin başında hedeflenen akademik ve teknik beklentileri karşılamıştır.

KAYNAKLAR

- **Apache Spark.** (2026). *MLlib: Main Guide - Spark 3.5.0 Documentation.* https://spark.apache.org/docs/latest/ml-guide.html adresinden erişildi.
- **Atalay, M. ve Çelik, E.** (2017). Büyük Veri Analizinde Artırılmış Hız: Apache Spark. *Gazi Mühendislik Bilimleri Dergisi*, 3(3), 167-178.
- **Erdem, M.** (2017). *Büyük Veri Analitiğinde Apache Spark ve Makine Öğrenmesi Yaklaşımları.* (Yüksek Lisans Tezi/Makale).
- **Kaba, A. ve Bağdatlı Kalkan, S.** (2022). Makine Öğrenmesi Algoritmaları ile Müşteri Terk Analizi: Bankacılık Sektörü Üzerine Bir Uygulama. *İşletme Araştırmaları Dergisi*.
- **Kaggle.** (2026). *Bank Customer Churn Prediction Dataset.* https://www.kaggle.com/ adresinden erişildi.
- **Zaharia, M., et al.** (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56-65.